AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Arkadiusz Górski

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Cel główny i motywacja realizacji projektu 3
1.2. Założenia projektowe i oczekiwane rezultaty 3
1.3. Podsumowanie założeń projektowych 4
1.4. Krótki opis działania listy dwukierunkowej 4
1.4.1. Schemat graficzny struktury 5

2. Analiza problemu **6**

2.1. Zastosowanie 6
2.2. Sposób działania programu i algorytmów 8
2.3. Pokazanie działania algorytmu na przykładzie 13

3. Projektowanie **15**

3.1. Narzędzia użyte w projekcie 15
3.2. Kilka słów o Doxygenie 16
3.3. GitHub i zdalne zarządzanie projektem 16
3.3.1. Co to jest GitHub? 16
3.3.2. Dlaczego warto go używać? 17
3.3.3. Podstawowe komendy Gita używane w projekcie 17
3.3.4. Dokumentacja wizualna projektu (Zrzuty ekranu) 17

4. Implementacja **18**

4.1. Weryfikacja działania w funkcji `main` 18
4.1.1. Przebieg scenariusza testowego i analiza operacji 19

5. Wnioski **22**

Literatura **23**

Spis rysunków **23**

Spis tabel **24**

Spis listingów **25**

1. Ogólne określenie wymagań

1.1. Cel główny i motywacja realizacji projektu

Głównym celem niniejszego projektu było zaprojektowanie, implementacja oraz szczegółowa weryfikacja poprawności struktury danych o charakterze dynamicznym, jaką jest dwukierunkowa lista liniowa (ang. Doubly Linked List), zrealizowana w języku programowania C++.

Współczesna inżynieria oprogramowania wymaga od programisty nie tylko znajomości gotowych rozwiązań oferowanych przez standardowe biblioteki (takie jak `std::list`), ale przede wszystkim pełnego zrozumienia mechanizmów niskopoziomowych. Projekt ten powstał w celu zgłębienia tajników zarządzania pamięcią dynamiczną na stercie, operowania na wskaźnikach oraz praktycznego zastosowania zaawansowanych wzorców projektowych, które w znaczący sposób podnoszą czytelność, skalowalność i modularność kodu źródłowego.

1.2. Założenia projektowe i oczekiwane rezultaty

W toku prac inżynieryjnych sformułowano następujące założenia funkcjonalne oraz niefunkcjonalne, które miały zostać zrealizowane

- Modułowa architektura kodu: Podział systemu na niezależne logicznie komponenty, w tym dedykowaną klasę węzła (`Node`), klasę odpowiedzialną za bezpieczną alokację (`NodeFactory`), strukturę iteratora umożliwiającą dwukierunkową nawigację (`ListIterator`) oraz klasę zarządzającą główną logiką struktury (`DoublyLinkedList`)

- Bezpieczeństwo pamięciowe: Zapewnienie pełnej kontroli nad cyklem życia obiektów w pamięci RAM poprzez jawną implementację destruktorów oraz operacji czyszczących, co pozwala na całkowite wyeliminowanie zjawiska wycieków pamięci

- Optymalizacja złożoności obliczeniowej. Osiignięcie stałej złożoności czasowej $O(1)$. dla operacji dodawania oraz usuwania elementów na obu końcach listy (`pushfront`), (`pushback`), (`popfront`), (`popback`).

1.3. Podsumowanie założeń projektowych

Poniższa tabela przedstawia zestawienie kluczowych założeń funkcjonalnych, technicznych i inżynieryjnych przyjętych na etapie koncepcyjnym projektu.

Tab. 1.1. Macierz założeń i zakresu projektu.

Kategoria / Aspekt	**Założenie / Wymaganie**	**Krótki opis realizacji w projekcie**
Architektura kodu	Modułowość i separacja odpowiedzialności	Podział systemu na niezależne komponenty: klasę węzła (`Node`), fabrykę (`NodeFactory`), iterator (`ListIterator`) oraz główny kontener listy.
Zarządzanie pamięcią	Niskopoziomowa kontrola sterty	Wykorzystanie manualnych operatorów `new/delete` w celu pełnego zrozumienia cyklu życia obiektów i eliminacji wycieków pamięci.
Wydajność algorytmiczna	Optymalizacja operacji brzegowych	Zapewnienie stałej złożoności czasowej $O(1)$ dla operacji wstawiania i usuwania elementów z obu końców listy.
Narzędzia inżynieryjne	Wersjonowanie i automatyczna dokumentacja	Stosowanie systemowej kontroli wersji (Git) do śledzenia historii zmian oraz generatora Doxygen do dokumentowania kodu.

1.4. Krótki opis działania listy dwukierunkowej

Lista dwukierunkowa to liniowa struktura danych składająca się z elementów zwanych węzłami (*Node*). W przeciwieństwie do listy jednokierunkowej, każdy węzeł w liście dwukierunkowej przechowuje dwie wartości wskaźnikowe:

- `next` – wskaźnik na następny węzeł w sekwencji,
- `prev` – wskaźnik na poprzedni węzeł w sekwencji.

Oprócz samego wskaźnika, węzeł przechowuje właściwe dane (np. wartość typu `int`). Struktura listy utrzymuje dwa kluczowe wskaźniki kontrolne: `head` (skierowany na pierwszy element) oraz `tail` (skierowany na ostatni element).

Dzięki obecności powiązań w obu kierunkach lista umożliwia efektywne przeglądanie danych zarówno od początku do końca, jak i od końca do początku. Dodatkowo operacje usuwania lub wstawiania elementów, gdy dysponujemy bezpośrednim wskaźnikiem na węzeł, mają złożoność czasową $O(1)$.

1.4.1. Schemat graficzny struktury

Poniższy graf przedstawia budowę listy dwukierunkowej z zaznaczonymi wskaźnikami `head` i `tail` oraz powiązaniami między węzłami.



Rys. 1.1. Graf

2. Analiza problemu

2.1. Zastosowanie

Struktura listy dwukierunkowej znajduje zastosowanie wszędzie tam, gdzie kluczowe jest sprawne poruszanie się w obu kierunkach (w przód i w tył) oraz błyskawiczna modyfikacja elementów bez konieczności przesuwania całej kolekcji. W przeciwieństwie do tradycyjnych tablic, pozwala ona na dodawanie i usuwanie węzłów ze środka, początku lub końca struktury w stałym czasie, o ile znamy wskaźnik na dany węzeł. Ze względu na tę unikalną charakterystykę, algorytm ten jest powszechnie wykorzystywany w systemach operacyjnych, bazach danych oraz aplikacjach codziennego użytku

Pamięć podręczna LRU

- Gdzie: W bazach danych (np. Redis, silniki SQL), pamięci podręcznej przeglądarek internetowych oraz pamięci podręcznej dysków
- Dlaczego: Gdy pamięć cache się zapełni, trzeba usunąć element najdawniej używany i szybko przenieść nowo pobrany element na sam początek. Lista dwukierunkowa pozwala na wypięcie węzła z dowolnego miejsca i wpięcie go na początek w stałym czasie

Historia przeglądarek (Przyciski „Wstecz" i „Dalej")

- Gdzie: W przeglądarkach internetowych (Chrome, Firefox, Safari) oraz w eksploratorach plików (np. Eksplorator Windows, Finder)
- Dlaczego: Użytkownik może przemieszczać się liniowo w przód i w tył między odwiedzonymi stronami lub folderami. Każda strona stanowi węzeł połączony z poprzecznicą i następnikiem.

Mechanizm Cofnij / Ponów

- Gdzie: W edytorach tekstu (Word, VS Code), programach graficznych (Photoshop, Figma) oraz arkuszach kalkulacyjnych.
- Dlaczego: Każda zmiana wprowadzona przez użytkownika tworzy nowy stan (węzeł). Lista pozwala na wycofywanie zmian aż do wskazanego punktu w historii oraz na ponowne ich aplikowanie.

Kolejki zadań i harmonogram procesów (OS Scheduler)

- Gdzie: W jądrze systemów operacyjnych (np. w systemie Linux przy zarządzaniu procesami).
- Dlaczego: Procesy oczekujące na przydział czasu procesora (CPU) są układane w kolejki. Jeśli proces zostanie anulowany lub zmieni priorytet, lista dwukierunkowa umożliwia natychmiastowe usunięcie go ze środka kolejki bez naruszania pozostałych procesów.

Odtwarzacze multimedialne (Playlisty)

- Gdzie: W aplikacjach muzycznych i wideo (np. Spotify, YouTube, VLC Player)
- Dlaczego: Umożliwia sprawne przechodzenie do następnego**next** lub poprzedniego **previous** utworu. Łatwo też zrealizować zapętlanie (tworząc z niej listę cykliczną) lub przepinanie kolejności utworów na liście

Zarządzanie stertą pamięci

- Gdzie: W niskopoziomowych mechanizmach alokacji pamięci RAM (np. w funkcjach malloc/free w języku C)
- Dlaczego: Bloki wolnej pamięci na stercie połączone są listą dwukierunkową. Przy zwalnianiu pamięci system może od razu sprawdzić poprzedni i następny blok, aby w czasie stałym połączyć je w jeden większy, spójny blok wolnej pamięci

Nawigacja w interfejsach użytkownika (GUI / Slajdery)

- Gdzie: W karuzelach zdjęć na stronach WWW, komponentach typu stepper (formularze wielokrokowe) oraz powiadomieniach ekranowych
- Dlaczego: Pozwala płynnie przełączać się między widokami, slajdami lub etapami bez konieczności przeliczania indeksów całej tablicy

lgorytmy przetwarzania tekstów i bufory edycyjne (Gap Buffer)

- Gdzie: W silnikach edytorów kodu i przetworników tekstu
- Dlaczego: Podczas pisania ciągłe wstawianie tekstu w środku wielkiego pliku byłoby bardzo powolne na tablicy. Podział tekstu na bloki połączone listą dwukierunkową ułatwia szybkie wstawianie i usuwanie znaków w miejscu kursora

2.2. Sposób działania programu i algorytmów

Działanie aplikacji opiera się na ścisłej współpracy trzech głównych komponentów: struktury danych (listy dwukierunkowej), wzorca Fabryki (odpowiedzialnego za bezpieczne powstawanie obiektów) oraz wzorca Iteratora (umożliwiającego niezależną i bezpieczną nawigację).

1. Inicjalizacja i tworzenie węzłów (Wzorzec Fabryki)

Problem: Bezpośrednie tworzenie obiektów węzłów za pomocą operatora new w metodach listy wiązałoby logikę biznesową z alokacją pamięci

Algorytm działania:

- Gdy użytkownik wywołuje metodę dodającą element do listy (np. add(data)), lista nie alokuje pamięci samodzielnie.
- Zamiast tego wysyła żądanie do klasy fabrycznej (NodeFactory): NodeFactory::createnode(data).
- Fabryka tworzy nową instancję węzła, przypisuje do niej przekazane dane oraz inicjalizuje wskaźniki next i prev jako puste (nullptr).

Gotowy węzeł jest zwracany do listy, która włącza go w strukturę

Podgląd przykładowego wzorca fabryki:

```cpp
#include <iostream>
#include <memory>
#include <string>

// 1. Interfejs produktu (Klasa bazowa)
class Product {
public:
    virtual ~Product() = default;
    virtual std::string operation() const = 0;
};

// 2. Konkretne produkty
class ConcreteProductA : public Product {
public:
    std::string operation() const override {
        return "Wynik dzialania Produktu A";
    }
};

class ConcreteProductB : public Product {
public:
    std::string operation() const override {
        return "Wynik dzialania Produktu B";
    }
}

// 3. Klasa Fabryki (Tworca)
class Factory {
public:
    enum class ProductType {
        TypeA,
        TypeB
    };

    // Metoda wytworcza (Factory Method)
    static std::unique_ptr<Product> createProduct(ProductType type)
    {
        switch (type) {
            case ProductType::TypeA:
                return std::make_unique<ConcreteProductA>();
            case ProductType::TypeB:
                return std::make_unique<ConcreteProductB>();
```

```
            default:
                return nullptr;
        }
    }
};

// 4. Uzycie struktury (Main)
int main() {
    // Tworzenie produktu A przez fabrykĘ
    std::unique_ptr<Product> prodA = Factory::createProduct(Factory
    ::ProductType::TypeA);
    if (prodA) {
        std::cout << prodA->operation() << std::endl;
    }

    // Tworzenie produktu B przez fabrykĘ
    std::unique_ptr<Product> prodB = Factory::createProduct(Factory
    ::ProductType::TypeB);
    if (prodB) {
        std::cout << prodB->operation() << std::endl;
    }

    return 0;
}
```

Listing 1. Wzorzec fabryki

1. Nawigacja i przeglądanie danych (Wzorzec Iteratora)

Problem: Kod zewnętrzny (klient) nie powinien mieć bezpośredniego dostępu do wskaźników head, tail ani struktury węzłów, aby uniknąć przypadkowego przerwania wiązań w liście.

Algorytm działania iteratora:

- Klient wywołuje metodęktóra zwraca obiekt iteratora ustawiony domyślnie na początek listy (head)
- Następnie sprawdzana jest metoda, czy bieżący węzeł ma następcę

```
        current != nullptr

```

- Jeśli tak, iterator zwraca dane z bieżącego węzła, a następnie przesuwa wskaźnik wewnętrzny

```
        current = current->next

```

Przejście w tył

- Sprawdzana jest metoda, czy istnieje poprzednik

```
        current->prev != nullptr

```

- Jeśli tak, iterator zwraca dane i przesuwa wskaźnik wstecz

```
       current = current->prev

```

Dzięki temu mechanizmowi użytkownik przegląda kolekcję sekwencyjnie, nie mając pojęcia o tym, jak fizycznie alokowane i łączone są węzły w pamięci

Oto przykładowy wzorzec iteratora

```cpp
#include <iostream>
#include <vector>

class Iterator {
    const std::vector<int>& dane;
    size_t indeks = 0;
public:
    Iterator(const std::vector<int>& d) : dane(d) {}
    bool maDalsze() const { return indeks < dane.size(); }
    int nastepny() { return dane[indeks++]; }
};

int main() {
    std::vector<int> liczby = {10, 20, 30};
    Iterator it(liczby);

    while (it.maDalsze()) {
        std::cout << it.nastepny() << " ";
    }
}
```

Listing 2. Wzorzec iteratora

2.3. Pokazanie działania algorytmu na przykładzie

Poniższy przykład demonstruje, jak w praktyce korzystać z przygotowanej struktury listy dwukierunkowej, współpracującej z fabryką węzłów (NodeFactory) oraz iteratorem dwukierunkowym (Iterator)

```
#include <iostream>
#include "DoublyLinkedList.h"
#include "NodeFactory.h"

int main() {
    // 1. Inicjalizacja listy
    DoublyLinkedList<int> list;

    // 2. Dodawanie elementow (pod spodem wykorzystywana jest Fabryka Wezlow)
    list.add_back(10);
    list.add_back(20);
    list.add_back(30);
    list.add_front(5); // Lista wyglada teraz tak: 5 <-> 10 <-> 20 <-> 30

    std::cout << "Rozmiar listy: " << list.get_size() << "\n\n";

    // 3. Uzycie Iteratora do przejscia w przod
    std::cout << "Iteracja w przod (od poczatku do konca):\n";
    auto it = list.get_iterator();
    while (it->has_next()) {
        std::cout << it->next() << " ";
    }
    std::cout << "\n\n";

    // 4. Uzycie Iteratora do przejscia w tyl
    std::cout << "Iteracja w tyl (od konca do poczatku):\n";
    // Ustawiamy iterator na koniec listy
    auto rev_it = list.get_reverse_iterator();
    while (rev_it->has_previous()) {
        std::cout << rev_it->previous() << " ";
    }
    std::cout << "\n";

    return 0;
}
```

Listing 3. Krótki przykład

Krótkie wyjaśnienie

1. Tworzenie listy i wstrzykiwanie przez fabrykę. Kiedy wywołujemy list.add_back(10), lista nie tworzy węzła bezpośrednio. Zgłasza się do klasy NodeFactory, która bezpiecznie alokuje pamięć i zwraca gotowy obiekt Node

2. Iteracja w przód (has_next() / next()): Iterator pobiera początkowy węzeł (head) i w pętli wypisuje wartość, a następnie bezpiecznie przesuwa się wskaźnikiem na kolejny element, nie ujawniając użytkownikowi struktury wskaźnikowej

3. Iteracja w tył (has_previous() / previous()): Wykorzystując dwukierunkowość struktury, iterator ustawiony na końcu (tail) pozwala na płynne cofanie się od ostatniego elementu do pierwszego

4. Oczekiwany wynik w konsoli

```
    Rozmiar listy: 4

Iteracja w przod (od poczatku do konca):
5 10 20 30

Iteracja w tyl (od konca do poczatku):
30 20 10 5
```

3. Projektowanie

3.1. Narzędzia użyte w projekcie

Podczas realizacji projektu wykorzystano nowoczesny, profesjonalny zestaw narzędzi wspierających pisanie kodu, kontrolę wersji, tworzenie dokumentacji technicznej oraz redagowanie raportu końcowego

- Visual Studio Code

  Główny edytor kodu źródłowego i środowisko pracy. Dzięki lekkiej konstrukcji oraz bogatemu ekosystemowi rozszerzeń (m.in. wsparcie dla C++, GitLens czy narzędzi do obsługi Markdown/LaTeX) umożliwił szybkie pisanie, debugowanie i organizację plików projektu

- GitHub

  System kontroli wersji używany do śledzenia zmian w kodzie źródłowym i plikach dokumentacji. Pozwolił na bezpieczne eksperymentowanie z kodem, tworzenie historii zmian oraz tworzenie kopii zapasowych projektu

- LaTeX

  System składu tekstowego wykorzystany do przygotowania oficjalnej, formalnej dokumentacji projektowej w formacie PDF. Zapewnił profesjonalną typografię, idealne formatowanie wzorów matematycznych, tabel oraz automatyczne generowanie spisu treści

- Doxygen

  Zaawansowane narzędzie służące do automatycznego generowania dokumentacji bezpośrednio z kodu źródłowego

3.2. Kilka słów o Doxygenie

Jak działa w projekcie: Doxygen analizuje specjalnie sformatowane komentarze w kodzie (np. w stylu JavaDoc/Javadoc-style: /// lub /*! ... */) umieszczone nad klasami, metodami i polami.

Co generuje: Na tej podstawie automatycznie tworzy przejrzystą dokumentację w formacie HTML oraz LaTeX.

Kluczowe zalety:

Tworzy hierarchię klas i struktur (pokazując relacje między Listą, Węzłem, Fabryką i Iteratorem).

Generuje grafy zależności i wywołań (call graphs), co ułatwia zrozumienie, jak poszczególne metody współpracują ze sobą.

Dba o spójność – jeśli zmienisz coś w kodzie i zaktualizujesz komentarz, dokumentacja API odzwierciedli to po ponownym uruchomieniu generatora.

3.3. GitHub i zdalne zarządzanie projektem

3.3.1. Co to jest GitHub?

GitHub to oparta na chmurze platforma hostingowa dla projektów programistycznych, która wykorzystuje system kontroli wersji **Git**. Pozwala na przechowywanie kodu źródłowego zdalnie, śledzenie każdej wprowadzonej zmiany oraz współpracę nad projektem. Działa jak zaawansowany "chmurowy dysk z historią wersji", w którym każda modyfikacja jest dokładnie opisana i przypisana do konkretnego autora i momentu w czasie.

3.3.2. Dlaczego warto go używać?

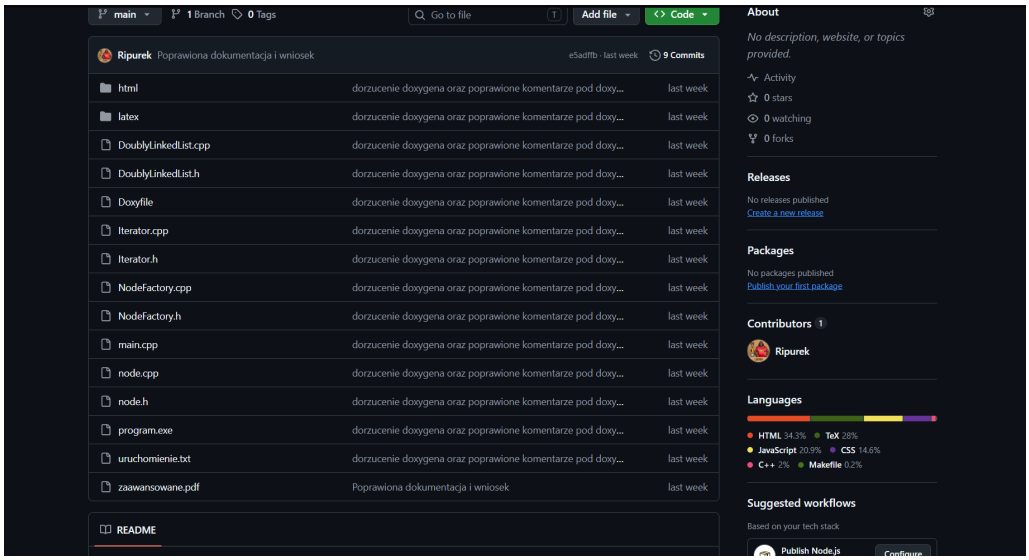
- **Bezpieczeństwo i kopia zapasowa:** Kod nie zniknie w przypadku awarii komputera, ponieważ jego pełna historia jest bezpiecznie zsynchronizowana w chmurze.
- **Historia i audytowalność zmian:** Możliwość powrotu do dowolnego wcześniejszego stanu projektu, sprawdzenie, co i kiedy zostało zmodyfikowane.
- **Przejrzystość struktury:** Czysty interfejs webowy pozwala łatwo przejrzeć pliki projektu, podglądnąć kod czy przeczytać plik
- **Portfolio inżynierskie:** Publiczne lub prywatne repozytorium stanowi dowód realizacji projektu i umiejętności korzystania z nowoczesnych narzędzi programistycznych.

3.3.3. Podstawowe komendy Gita używane w projekcie

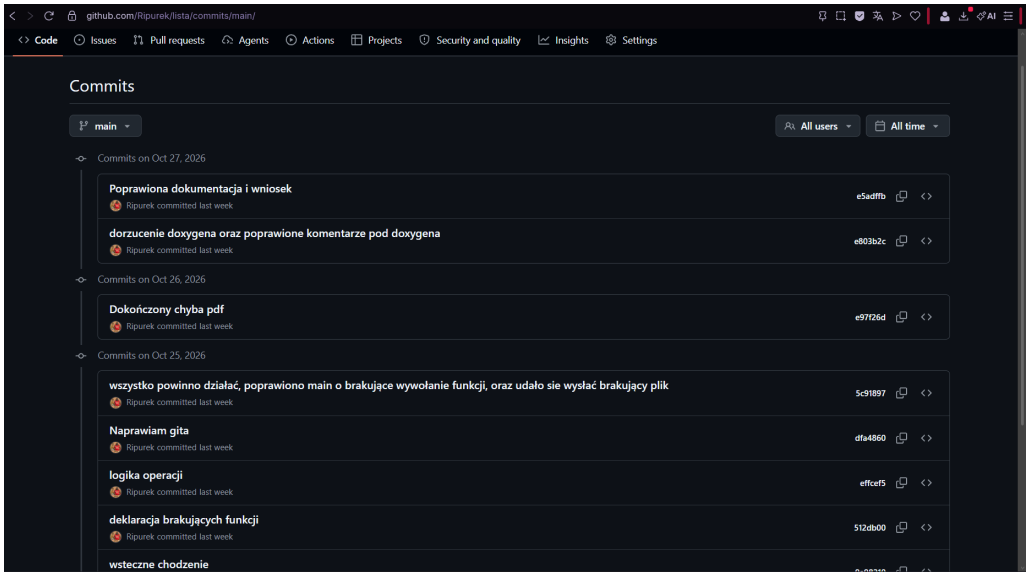
Podczas pracy nad projektem najczęściej korzystano z następujących poleceń w terminalu:

- `git clone <url>` – pobranie (sklonowanie) zdalnego repozytorium na lokalny komputer.
- `git status` – sprawdzenie aktualnego stanu plików (które zostały zmienione, a które jeszcze nie zostały dodane do śledzenia).
- `git add .` – dodanie zmienionych plików do strefy przygotowawczej (*staging area*) przed zapisem.
- `git commit -m "Opis zmian"` – trwały zapis (*commit*) przygotowanych zmian w lokalnej historii z krótkim opisem wykonanej pracy.
- `git push` – wysłanie zapisanych lokalnie commitów na zdalny serwer (GitHub).
- `git pull` – pobranie najnowszych zmian z serwera (przydatne przy pracy na wielu urządzeniach).

3.3.4. Dokumentacja wizualna projektu (Zrzuty ekranu)



Rys. 3.1. Widok struktury katalogów i plików projektu w zdalnym repozytorium GitHub.



Rys. 3.2. Historia commitów ilustrująca etapowy rozwój projektu.

4. Implementacja

4.1. Weryfikacja działania w funkcji `main`

W celu potwierdzenia poprawności działania zaimplementowanych struktur danych oraz wzorców projektowych przygotowano kompleksowy scenariusz testowy w funkcji `main`. Testy weryfikują nie tylko poprawność przydziału i zwalniania pamięci, ale również bezbłędne zarządzanie wskaźnikami dwukierunkowymi przy dynamicznych modyfikacjach struktury oraz poprawność działania iteratorów w obu kierunkach.

4.1.1. Przebieg scenariusza testowego i analiza operacji

1. **Inicjalizacja i dodawanie elementów do listy (`push_back`, `push_front`):** Na początku wywoływana jest funkcja `push_back` dla wartości 10, 20 oraz 34, a następnie `push_front` dla wartości 4. Każda z tych operacji przekazuje zadanie utworzenia węzła do klasy `NodeFactory::createNode`. Test sprawdza, czy po wykonaniu tych operacji lista prawidłowo łączy skrajne węzły oraz czy głowa (`head`) i ogon (`tail`) wskazują na właściwe obiekty.

2. **Iteracja w przód za pomocą iteratora (`begin()`, `end()`):** Pętla `for` wykorzystuje metodę `list.begin()` do pobrania iteratora na pierwszy element oraz `list.end()` jako warunek stopu. W każdej iteracji używany jest przeciążony operator inkrementacji (`++it`) przesuwający wskaźnik na `next`, oraz operator wyłuskania (`*it`) pobierający wartość danych.

3. **Wstawianie elementów pod dowolny indeks (`insert_at`):** Przetestowano wstawienie wartości 15 pod indeks 1 oraz wartości 17 pod indeks 0. Operacja ta weryfikuje poprawność aktualizacji aż czterech wskaźników jednocześnie (`next` i `prev` dla węzła poprzedzającego, nowego oraz następującego), co jest kluczowe dla zachowania spójności listy dwukierunkowej.

4. **Iteracja wsteczna (`rbegin()`, `rend()`):** Dzięki dwukierunkowej strukturze węzłów przetestowano możliwość przechodzenia listy od końca do początku. Wykorzystano do tego metody `rbegin()` (ustawiającą iterator na wskaźnik `tail`) oraz `rend()` (`nullptr`), operując dekrementacją (`--it`) przesuwającą wskaźnik wstecz po `prev`.

5. **Usuwanie skrajnych elementów (`pop_front`, `pop_back`):** Wywołanie metod odpinających elementy z początku i końca listy sprawdza, czy wskaźniki `head` i `tail` są poprawnie przestawiane na sąsiednie węzły, a usuwane obiekty zwalniane z pamięci operacyjnej RAM za pomocą operatora `delete`.

6. **Test nawigacji względnej iteratora (`++it`, `--it`):** Ostatnia część scenariusza weryfikuje bezpośredni skok iteratora do przodu i powrót do poprzedniego elementu. Sprawdza to, czy iterator zachowuje spójny stan wewnętrzny i pozwala na swobodne przemieszczanie się po strukturze w dowolnym momencie.

```cpp
#include <iostream>
#include "DoublyLinkedList.h"

int main() {
    DoublyLinkedList list;  /// Tworzymy pusta liste

    list.push_back(10); /// Wrzucamy liczbe 10 (tworzy sie wezel przez fabryke)
    list.push_back(20);  /// Wrzucamy liczbe 20
    list.push_back(34);  /// Wrzucamy liczbe 34
    list.push_front(4); /// Wrzucamy liczbe 4 na poczatek listy

    std::cout << "Wyswietl cala liste: ";
    for (auto it = list.begin(); it != list.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << "\n\n";

    /// 2. Dodaj element pod wskazany indeks
    std::cout << "Wstawiam 15 pod indeks 1:\n";
    list.insert_at(1, 15); /// Lista: 10, 15, 20, 30
    std::cout << "Lista: ";
    for (auto it = list.begin(); it != list.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << "\n\n";

    std::cout << "Wstawiam 17 pod indeks 0:\n";
    list.insert_at(0, 17); /// Lista: 10, 15, 20, 30
    std::cout << "Lista: ";
    for (auto it = list.begin(); it != list.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << "\n\n";

    /// 3. Wyswietl liste w odwrotnej kolejnosci
    std::cout << "Wyswietl liste w odwrotnej kolejnosci: ";
    for (auto it = list.rbegin(); it != list.rend(); --it) {
        std::cout << *it << " ";
    }
    std::cout << "\n\n";

    /// 4. Usuwanie elementow
    std::cout << "Usuwam element z poczatku i z konca\n";
    list.pop_front(); /// Usuwa 10
```

```
45      list.pop_back();  /// Usuwa 30
46      std::cout << "Lista po usunieciu: ";
47      for (auto it = list.begin(); it != list.end(); ++it) {
48          std::cout << *it << " ";
49      }
50      std::cout << "\n\n";
51
52      /// 5. Test wyswietlania nastepnego i poprzedniego elementu
    wzgledem iteratora
53      list.push_back(40); /// Lista ma teraz: 15, 20, 40
54      auto it = list.begin(); /// Stoimy na 15
55      std::cout << "Aktualny element (iterator): " << *it << "\n";
56      ++it; /// Przechodzimy do nastepnego
57      std::cout << "Wyswietl nastepny element: " << *it << "\n";
58      --it; /// Wracamy do poprzedniego
59      std::cout << "Wyswietl poprzedni element: " << *it << "\n\n";
60
61      return 0;
62  }
```

Listing 4. Pokazanie jak dziala w main

5. Wnioski

Realizacja niniejszego projektu pozwoliła na praktyczne zapoznanie się ze strukturą listy dwukierunkowej oraz wzorcami projektowymi Fabryki i Iteratora. Na podstawie przeprowadzonych prac i testów wyciągnięto następujące wnioski:

- **Ciekawy i użyteczny zamysł architektoniczny:** Połączenie dwukierunkowej struktury danych z wzorcem Fabryki i Iteratora to niezwykle praktyczne rozwiązanie. Wykorzystanie Fabryki do tworzenia węzłów gwarantuje pełną izolację alokacji pamięci od logiki listy, natomiast Iterator udostępnia intuicyjny i bezpieczny interfejs do poruszania się po kolekcji (zbliżony do standardowych bibliotek, takich jak STL w C++).
- **Wymagający proces zrozumienia i pierwszej implementacji:** Pierwszorazowa próba zaprojektowania i prawidłowego zaimplementowania tej struktury wymaga znacznego nakładu czasu. Zrozumienie relacji między wskaźnikami (`head`, `tail`, `next`, `prev`), poprawna obsługa przypadków brzegowych (np. pusta lista, wstawianie na brzegach) oraz prawidłowa obsługa wycieków pamięci i przeciążanie operatorów iteratora stanowimy spore wyzwanie na etapie nauki.
- **Kluczowa rola bieżącego dokumentowania prac:** Jednym z najważniejszych wniosków organizacyjnych jest konieczność tworzenia dokumentacji technicznej oraz komentarzy Doxygen na bieżąco w trakcie pisania kodu. Odłożenie przygotowywania dokumentacji na sam koniec projektu skutkuje nakładaniem się ogromnej liczby szczegółów do uzupełnienia, co wymaga przypominania sobie zawiłości zaimplementowanych algorytmów i zajmuje znacznie więcej czasu niż stopniowa opisywanie powstałych modułów.

Podsumowując, mimo początkowej trudności w zrozumieniu i zaimplementowaniu wszystkich zależności, powstały program jest elastyczną, bezpieczną i wydajną strukturą danych o szerokim zastosowaniu inżynierskim.

Spis rysunków

1.1. Graf 5

3.1. Widok struktury katalogów i plików projektu w zdalnym repozytorium GitHub. 18

3.2. Historia commitów ilustrująca etapowy rozwój projektu. 18

Spis tabel

1.1. Macierz założeń i zakresu projektu. 4

Spis listingów

1. Wzorzec fabryki 9
2. Wzorzec iteratora 12
3. Krótki przykład 13
4. Pokazanie jak dziala w main 20